



ASCII

PROSEDUR DAN FUNGSI



Prosedur dan Fungsi

Dasar Teori

Prosedur dan fungsi berguna agar saat ada kumpulan kode tertentu yang digunakan secara berulang tidak perlu ditulis secara repetitif sehingga kode yang ditulis lebih rapi dan pendek. Prosedur adalah blok kode program yang dirancang untuk menjalankan serangkaian tugas atau aksi tertentu **tanpa** mengembalikan nilai (*return value*) kepada pemanggilnya. Prosedur digunakan ketika kita hanya ingin program (seperti untuk menampilkan output).

Fungsi adalah blok kode program yang dirancang untuk memproses data dan **mengembalikan** sebuah nilai (*return value*) kepada pemanggilnya. Hasil dari fungsi ini biasanya ditampung ke dalam sebuah variabel atau langsung digunakan dalam operasi perhitungan lain.

Prosedur	Fungsi
Tidak terdapat tipe data dalam deklarasi. Dideklarasikan dengan keyword Void	Terdapat tipe data dalam deklarasi seperti int, float, string, dll
Tidak dapat mengembalikan nilai	Dapat Mengembalikan nilai

Deklarasi Prosedur dan Fungsi

Saat mendeklarasikan fungsi disarankan penggunaan nama yang mudah diingat serta jelas agar mudah digunakan dan dibaca Pastikan fungsi yang ada juga benar secara penempatan agar tidak terjadi error saat menjalankan program..



Deklarasi Prosedur:

```
void nama_parameter(tipe_data_parameter nama_parameter) {  
    // isi fungsi  
} :
```

Prosedur tidak harus menggunakan parameter dan prosedur tidak bisa mengembalikan nilai.

Deklarasi Fungsi:

```
Tipe_data nama_fungsi(tipe_data_parameter nama_parameter){  
    // isi fungsi  
}
```

Fungsi dapat mengembalikan nilai dan wajib menggunakan parameter.

Jenis Penulisan Prosedur dan Fungsi

Pada program prosedur atau fungsi dapat ditaruh sebelum atau sesudah penulisan fungsi **main()**. Bila ditulis setelah fungsi **main()** maka perlu didefinisikan terlebih dahulu Pemilihan posisi ini bebas dilakukan sesuai dengan keinginan penulis kode, tetapi jika di tulis di atas fungsi **main()** maka tidak perlu menulisnya kembali dibawahnya..



Contoh penulisan prosedur atau fungsi sebelum fungsi **main()**:

```
#include <iostream>
using namespace std;

// Fungsi ditaruh sebelum main
int kuadrat(int x) {
    return x * x;
}

// Prosedur ditulis sebelum main
void sapa(string nama) {
    cout << "Halo, " << nama << "!" << endl;
}

int main() {
    sapa("Budi");
    cout << "Hasil 5 kuadrat: " << kuadrat(5) << endl;
    return 0;
}
```



Contoh penulisan prosedur atau fungsi setelah **main()**:

```
#include <iostream>
using namespace std;

// Definisikan prosedur atau fungsi sebelum fungsi main
int kuadrat(int x);
void sapa(string nama);

int main() {
    sapa("Andi");
    cout << "Hasil 3 kuadrat: " << kuadrat(3) << endl;
    return 0;
}

// Penulisan fungsi setelah main
int kuadrat(int x) {
    return x * x;
}

// Penulisan prosedur setelah main
void sapa(string nama) {
    cout << "Halo, " << nama << "!" << endl;
}
```



Kegunaan Prosedur dan Fungsi

1. Memecah program menjadi sub-program yang lebih sederhana agar tidak menumpuk.
2. Meningkatkan efisiensi karena kode dapat dipanggil kembali (*reusable*).
3. Memudahkan pemahaman logika program yang semakin kompleks

Parameter dan Argumen

Parameter adalah variabel yang digunakan dalam deklarasi fungsi atau prosedur untuk menerima input. Sedangkan argumen adalah nilai yang dikirimkan saat fungsi atau prosedur dipanggil.

Contoh parameter pada prosedur:

```
#include <iostream>
using namespace std;

// Bagian Definisi Prosedur
// Di sini 'namaHero', 'level', dan 'hp' adalah PARAMETER.
// Mereka hanyalah "variabel kosong" yang menunggu nilai.
void tampilkanStatus(string namaHero, int level, double hp) {
    cout << "≡ STATUS HERO ≡" << endl;
    cout << "Nama : " << namaHero << endl;
    cout << "Level : " << level << endl;
    cout << "Darah : " << hp << "%" << endl;
    cout << "===== " << endl;
}
```



```
    cout << endl;
}

int main() {
    // Bagian pengambilan argumen
    // Kasus 1: Mengirim nilai secara langsung
    // "Alucard", 15, dan 90.5 adalah argumen.
    tampilkanStatus("Alucard", 15, 90.5);

    // Kasus 2: Mengirim menggunakan variabel lain
    string hero2 = "Miya";
    int lvl2 = 12;
    double hp2 = 45.0;

    // Variabel 'hero2', 'lvl2', dan 'hp2' dikirim sebagai argumen
    tampilkanStatus(hero2, lvl2, hp2);
    return 0;
}
```



Jenis Jenis Parameter Dalam C++

1. Parameter berdasarkan nilai (pass by Value)

Parameter berdasarkan nilai artinya argumen hanya disalin ke dalam fungsi dan tidak mempengaruhi variabel aslinya.

```
#include <iostream>
using namespace std;

// prosedur ini menerima nilai dari variable gajiSaya
void cobaUbahGaji(int gajiSaya) {
    gajiSaya = 1000000000; // gaji di dalam prosedur
    cout << "Gaji pas di dalam fungsi: " << gajiSaya << endl;
}

int main() {
    int gajiSaya = 5000000; // gaji di dalam main

    cout << "Gaji awal: " << gajiSaya << endl;

    cobaUbahGaji(gajiSaya); // kirim ke fungsi

    cout << "Gaji setelah fungsi dipanggil: " << gajiSaya << endl;

    return 0;
}
```




2. Parameter Berdasarkan Referensi (Pass by Reference)

Berbeda dengan parameter berdasarkan nilai, parameter berdasarkan referensi menggunakan &, sehingga perubahan dalam fungsi akan mempengaruhi variabel aslinya.

```
#include <iostream>
using namespace std;

// perhatikan tanda '&' di sebelah parameter
void tukarsenjata(string &senjata) {
    senjata = "Sarung Wadimor"; // langsung ganti isi variabel
    aslinya
}

int main() {
    string senjatahero = "Kayu Busuk";

    cout << "Senjata awal: " << senjatahero << endl;

    tukarsenjata(senjatahero); // panggil prosedur

    // variable dalam main akan ikut berubah
    cout << "Senjata sekarang: " << senjatahero << endl;

    return 0;
}
```



3. Parameter Berdasarkan Default Value

Parameter ini memiliki nilai default jika tidak diberikan argumen saat pemanggilan fungsi.

```
#include <iostream>
using namespace std;

// 'kadarGula' punya nilai default "Normal"
// parameter default wajib di taruh paling kanan!
void pesankopi(string menu, string kadargula = "Normal") {
    cout << "Pesanan: " << menu << " | Gula: " << kadargula << endl;
}

int main() {
    // Kasus 1: pengguna mengirim 1 parameter
    pesankopi("Kopi Susu");

    // Kasus 2: pengguna mengirim 2 parameter dan menggantikan
    // parameter default dari prosedur
    pesankopi("Americano", "Less Sugar");

    return 0;
}
```

Variable Global dan Lokal

Dalam mendeklarasikan sebuah variabel ketika membuat sebuah fungsi harus memperhatikan scope variabel yang dibuat. Scope variabel dibagi menjadi dua yaitu **variabel global** dan **variabel lokal**.



- **Variabel Global**

Variabel yang dideklarasikan di luar fungsi dan bisa digunakan di seluruh bagian program.

- **Variabel Lokal**

Variabel yang dideklarasikan di dalam fungsi dan hanya bisa digunakan dalam fungsi tersebut.

Contoh penerapan variabel global:

```
#include <iostream>
using namespace std;

// variabel global
int skorTertinggi = 100;

void cetakSkor() {
    // Fungsi ini akan membaca variabel global skorTertinggi
    cout << "Skor Fungsi: " << skorTertinggi << endl;
}

int main() {
    cout << "Skor : " << skorTertinggi << endl;

    // Panggil fungsi
    cetakSkor();

    return 0;
}
```



Contoh penerapan variabel lokal:

```
#include <iostream>
using namespace std;

void calonJuara() {
    // variabel lokal, cuma bisa dipakai di dalam fungsi ini
    string Team = "RRQ";

    cout << "Tim: " << Team << endl;
}

int main() {
    calonJuara(); // fungsi berjalan normal

    // namun jika kita coba akses Team di sini
    // cout << "Tim: " << Team << endl;
    // maka akan error karena Team tidak dikenal di sini

    return 0;
}
```

Jika ada variabel lokal dengan nama yang sama dengan variabel global, maka variabel lokal akan menimpa atau memprioritaskan variabel lokal ini disebut shadowing variabel.

Contoh penerapan shadowing variabel:

```
#include <iostream>
using namespace std;

// Variabel global bernilai 10
int angka = 10;

void cobaShadow() {
```



```
// variabel lokal akan menimpa variabel global 'angka' di dalam  
fungsi ini  
int angka = 500;  
  
cout << "Nilai angka dalam fungsi: " << angka << " (Ini yang  
lokal)" << endl;  
}  
  
int main() {  
    cout << "Nilai angka di global: " << angka << " (Ini yang  
global)" << endl;  
  
    cobaShadow(); // Panggil fungsi yang melakukan shadowing  
  
    // setelah fungsi angka akan kembali menggunakan variabel global  
    cout << "Nilai angka akhir: " << angka << " (Balik ke global)" <<  
endl;  
  
    return 0;  
}
```



Fungsi Overloading

Fungsi Overloading adalah teknik yang memungkinkan kita membuat beberapa fungsi dengan nama yang sama, tetapi dengan jumlah atau tipe parameter yang berbeda. Hal ini dapat dilakukan karena compiler akan secara otomatis memilih fungsi yang sesuai berdasarkan argumen yang diberikan saat pemanggilan.

Contoh penerapan fungsi overloading:

```
#include <iostream>
using namespace std;

// Fungsi menjumlahkan bilangan bulat
int tambah(int a, int b) {
    return a + b;
}

// Fungsi 2 menjumlahkan bilangan desimal
double tambah(double a, double b) {
    return a + b;
}

int main() {
    cout << "Hasil Int      : " << tambah(5, 6) << endl;
    cout << "Hasil Double : " << tambah(5.5, 6.6) << endl;
    return 0;
}
```

Keyword Return

Return dalam C++ memiliki dua fungsi utama dalam sebuah fungsi:

1. Mengembalikan nilai dari fungsi kepada pemanggilnya.
2. Menghentikan eksekusi fungsi secara langsung, meskipun masih ada kode di bawahnya.



Contoh penerapan return yang mengembalikan nilai:

```
#include <iostream>
using namespace std;

int hitungLuas(int panjang, int lebar) {
    int hasil = panjang * lebar;
    return hasil; // return mengembalikan nilai hasil
}

int main() {
    int p = 10;
    int l = 5;

    // pemanggilan fungsi
    int luasTanah = hitungLuas(p, l);

    cout << "Panjang: " << p << "m, Lebar: " << l << "m" << endl;

    // Sekarang variabel luasTanah sudah memiliki nilai yang
dikembalikan
    cout << "Total Luas Tanah: " << luasTanah << " meter persegi." <<
endl;

    return 0;
}
```

Contoh penerapan return untuk menghentikan eksekusi kode:



```
void cekGenap(int angka) {  
    if (angka % 2 == 0) {  
        cout << angka << " adalah Genap." << endl;  
        return; // Stop di sini dan kode di bawah tidak akan  
        dijalankan  
    }  
    // Kode ini hanya jalan kalau angka ganjil  
    cout << angka << " adalah Ganjil." << endl;  
}
```

Fungsi Rekursif

Rekursif adalah teknik dalam pemrograman di mana fungsi memanggil dirinya sendiri. Biasanya digunakan untuk masalah perulangan dan memerlukan kondisi agar berhenti.

Contoh tanpa rekursif (iteratif):

```
#include <iostream>  
using namespace std;  
  
int faktorialIteratif(int n) {  
    int hasil = 1;  
    for (int i = 1; i ≤ n; i++) {  
        hasil *= i;  
    }  
    return hasil;  
}  
  
int main() {  
    cout << "5! = " << faktorialIteratif(5) << endl;  
    return 0;  
}
```

Contoh dengan rekursif (rekursif):





```
#include <iostream>
using namespace std;

int faktorialRekursif(int n) {
    if (n == 1) return 1; // Base case (berhenti saat n = 1)
    return n * faktorialRekursif(n - 1); // Rekursi
}

int main() {
    cout << "5! = " << faktorialRekursif(5) << endl;
    return 0;
}
```

Aspek	Rekursif	Iteratif
Cara Kerja	Memanggil fungsi dirinya sendiri	Menggunakan perulangan (for / while).
Base Case	Diperlukan kondisi untuk menghentikan rekursi	Tidak diperlukan
Memori	Boros (karena banyak pemanggilan fungsi)	Hemat (langsung dieksekusi)
Kecepatan	Lebih lambat karena overhead pemanggilan fungsi	Lebih cepat karena hanya menggunakan loop
Kemudahan	Lebih intuitif untuk konsep matematika	Lebih mudah dipahami dalam pemrograman
Resiko Error	Bisa menyebabkan stack overflow jika n besar	Lebih stabil



File Header

Pada Pemrograman C++, untuk dapat menggunakan FUNGSI cout dan cin, kita memerlukan header iostream, ini berarti iostream merupakan pustaka fungsi. Header merupakan sebuah pustaka fungsi yang dapat kita pakai. Berikut beberapa header yang dapat kita pakai.

1. <iostream>

Merupakan singkatan dari input output stream header yang digunakan sebagai standar input output operasi yang digunakan oleh bahasa C++. Fungsi-fungsi yang ada di dalam iostream antara lain sebagai berikut.

Cout : Fungsi keluaran C++ yang menampilkan data dengan tipe data apapun ke layar atau terminal

Cin : Fungsi masukan pada c++ yang bisa memasukkan data berupa data numerik dan karakter

endl : Suatu Fungsi yang manipulator yang digunakan untuk melakukan perintah pindah baris

2. <conio>

Merupakan File Header yang berfungsi untuk menampilkan hasil antarmuka kepada pengguna. Fungsi -fungsi yang ada di dalam conio antara lain sebagai berikut :

getch() : Singkatan dari get character and echo yang digunakan untuk menahan (pause) output suatu program dan akan kembali mengeksekusi setelah kita melakukan inputan baik itu tombol enter atau tombol lainnya dan inputan tersebut tidak ditampilkan dalam window.



getche() : Secara fungsi sama dengan getch() akan tetapi ketika melakukan inputan, inputan tersebut tampil dalam window.

clrscr() : Singkatan dari clear screen yang digunakan untuk membersihkan layar windows.

3. <math.h>

Merupakan file header yang berfungsi untuk operasi matematika. Fungsi-fungsi yang ada di math.h antara lain sebagai berikut :

sqrt() : Fungsi ini digunakan untuk menghitung akar dari suatu bilangan. Bentuk umum penulisannya adalah sqrt(bilangan).

pow() : Fungsi ini digunakan untuk menghitung hasil dari perpangkatan suatu bilangan. Bentuk umum penulisannya adalah pow(bilangan, pangkat).

sin() : Fungsi ini digunakan untuk menghitung nilai sinus, cosinus, dan tangen dari sudut.

4. <stdlib.h>

Merupakan file header yang berfungsi untuk operasi pembandingan dan operasi konversi. Fungsi-fungsi yang ada di stdlib.h antara lain sebagai berikut :

Max() : Digunakan untuk menentukan nilai maksimum dari 2 bilangan yang diinputkan. Bentuk penulisannya adalah max(bilangan1, bilangan2);

Min() : Digunakan untuk menentukan nilai minimum dari 2 bilangan yang diinputkan.



Bentuk penulisannya adalah `min(bilangan1, bilangan2);`

`atof()` : Digunakan untuk mengkonversi nilai string menjadi bilangan bertipe double.

Bentuk umum penulisannya adalah `atof(char nama_variabel);`

`atoi()` : Digunakan untuk mengkonversi nilai string menjadi bilangan bertipe integer.

Bentuk umum penulisannya adalah `atoi(char nama_variabel);`

5. **<string.h>**

Merupakan file header yang berfungsi untuk melakukan manipulasi string. Fungsi-fungsi yang ada di `string.h` antara lain sebagai berikut :

`strcpy()` : Fungsi ini digunakan untuk menyalin suatu string ke variabel tujuan. Bentuk umum penulisannya adalah `strcpy(variabelTujuan, string);`

`strlen()` : Fungsi ini digunakan untuk menghitung jumlah karakter yang ada dalam suatu string. Bentuk umum penulisannya adalah `strlen(string);`

`strcmp()` : Fungsi ini digunakan untuk membandingkan 2 buah string. Bentuk umum penulisannya adalah `strcmp(string1, string2);`

`strrev()` : Fungsi ini digunakan untuk membalikan urutan suatu string. Bentuk umum penulisannya adalah `strrev(string);`

`strlwr()` : Fungsi ini digunakan untuk mengubah semua huruf menjadi huruf kecil. Bentuk umum penulisannya adalah `strlwr(string);`

`strupr()` : Fungsi ini digunakan untuk mengubah semua huruf menjadi huruf kapital. Bentuk umum penulisannya adalah `strupr(string);` .



`strcat()` : Fungsi ini digunakan untuk menggabungkan 2 buah string, untuk menggunakan fungsi ini juga harus menambahkan file header `ctype.h`.
Bentuk umum penulisannya adalah `strcat(variabelTujuan, string);`



Studi Kasus

Simulasikan pergerakan robot sederhana di koordinat kartesius (X, Y). Tantangannya, setiap robot bergerak, baterainya harus berkurang. Jika baterai habis, robot tidak boleh bergerak. Karena posisi (X, Y) dan Baterai harus berubah terus menerus, mahasiswa wajib menggunakan *Pass by Reference*.

1. Ilustrasikan bagaimana sistem itu dapat bekerja.
2. Implementasikan kode tersebut dalam sebuah program c++.